

Parsing, Code Generation & Optimization

Compilers

1. Consider the following command language:

$$C ::= L \mid R \mid F \mid B \mid C;C \mid C\|C \mid (C)$$

The meaning of these commands is not important for this assignment, but informally, **L** means “go left”, **R** means “go right”, **F** means “go forward”, **B** means “go backward”, $C_1;C_2$ executes C_1 followed by C_2 , and $C_1\|C_2$ non-deterministically executes C_1 or C_2

- Describe one way in which this grammar is ambiguous and how it could be modified to fix the ambiguity. *(There are multiple ways in which this grammar is ambiguous.)*
- Write a bottom-up parser for this grammar as a PDA (a pushdown automata). *(The general receipe is provided in `pdas.pdf`. It might help to think about what transitions are shift transitions and what transitions are reduce transitions.)*
- Show that the command $F;L\|R$ is accepted by your PDA by providing a sequence of instance descriptions that details all the steps. *(It might again help to think about what steps are shift steps and what steps are reduce steps.)*
- Explain where a shift/reduce ambiguity arises in your computation. *(A shift/reduce ambiguity is when either a shift or a reduce can be performed on the same instance description, while still getting the word to be accepted.)*

2. Consider the following Intermediate Representation (IR) code:

```
a = b
b = 2 * a
c = a
d = c + c
```

- Translate the above block to assembly code one instruction at a time, using 3 registers, showing how the register and address descriptors are updated.
- Optimize the above code, assuming that a, b, c, d are used in other blocks, and mentioning the names of the optimizations you have used.
- Describe the cost of the optimized code and compare it to the cost of the non-optimized code (no need to translate the optimized code to assembly too). Which version is the less costly? Justify your answer.